Tutorial on Sorting

Matteo Fasiolo

Intro to Sorting

The task of rearranging **items** in a **container** according to a pre-specified **order**.

- Items: Numbers, Text, Dates...
- Containers: Vectors, Lists, Tables...
- Order: Numerical, Alphabetical, Chronological...

Sorting is a frequently encountered task in computing.

- Rank students by their grades.
- Rank webpages by their relevance to user's search keywords.

Here we only talk about **sorting numbers in a vector**, in **ascending order**.

Usually, sorting algorithms are already implemented as a part of the programming language.

In R, you can do:

```r
v <- c(3,5,4,2)
sort(v)
```

```
[1] 2 3 4 5
```

```r
v <- c('matteo','bob','anthony')
sort(v)
```

```
[1] "anthony" "bob"     "matteo"
```

There are many existing sorting algorithms.

Basic ingredients

For our convenience, let us write a function that swaps two elements in a vector.

```
swap <- function(v, i, j){
 tmp <- v[i]
 v[i] <- v[j]
 v[j] <- tmp
 return(v)
}
```

```
v <- c(1,2,3,4)
swap(v, 1, 3)
```

```
[1] 3 2 1 4
```

Let's write a function that finds index of max element in vector:

```
find_max_idx <- function(v){
  len <- length(v)
  idx <- 1
  max <- v[1]
  if (len >= 2) {
    for (i in 2:len){
      if(v[i] > max){
        max <- v[i]
        idx <- i
      }
    }
  }
  return(idx)
}
v <- c(2,4,3,1)
find_max_idx(v)
```

```
[1] 2
```

How do you sort?

How do we use the functions above to find the n largest elements in an n vector?

Imagine something like this:

1. Find the largest,
2. Find the second largest,
3. ...
4. Find the n-th largest,

If we place each selected value into its appropriate position within a **new vector**, we obtain a vector in ascending order.

Can we reuse the same vector variable rather than constructing a separate output vector?

A Sorting Algorithm

Assume you are sorting a vector in ascending order.

1. Find the largest in [3,4,5,2], put it at the end.
   - Output is [3,4,2,5].
2. Find the 2nd largest in [3,4,2,5], put it at the second last.
   - Output is [3,2,4,5].
3. Find the 3rd largest in [3,2,4,5], put it at the third last.
   - Output is [2,3,4,5].
4. Done.

Sort 1.0: Pseudo Code

Input: Vector `v` with length `len`.

Output: Vector `v` in ascending order.

For i from 1 to `len-1`

1. Find maximum from the **unsorted part** of the vector with

   ```
   max_idx <- find_max_idx(v[1:(len-i+1)])
   ```

2. Swap it with the `i`-th last element using

   ```
   v <- swap(v, max_idx, len - i + 1)
   ```

Question: why does

```
find_max_idx(v[1:(len-i+1)])
```

find the maximum from the unsorted part of the vector?

Problem 1

See `tut_sort_template.R` file.

1. Based on the slides above, write functions
   - `find_max_idx(v)`
   - `swap(v, i, j)`
2. In `find_max_idx`, add a line to print out the following message:
   - "`The largest element is X`", replace `X` with the largest element. Hint: look at the output of the code `y <- 1; cat(sprintf("y = %d", y))`
3. In `swap` add a line of code to print out the following message:
   - "`I am swapping X1 with X2`", replace `X1` and `X2` with elements to be swapped.
4. Write test cases for your implementation of `find_max_idx` and `swap`. Make sure they are implemented correctly.

5. Implement the sort algorithm described above.
   - Use the `swap` and `find_max_idx` you just wrote.
   - At each iteration print out the vector after the swap.
6. From the printed text generated by your code, see if the sort algorithm works as you imagined.

Sort 2.0: A Recursive Sort

If you sort the following vector

```
v <- c(5,3,2,1,2,4)
sort(v)  # length(v) = 6
```

```
[1] 1 2 2 3 4 5
```

after each swap, the vector looks like

```
Iteration 0: 5 3 2 1 2 4
Iteration 1: 4 3 2 1 2 5
Iteration 2: 2 3 2 1 4 5
Iteration 3: 2 1 2 3 4 5
Iteration 4: 2 1 2 3 4 5
Iteration 5: 1 2 2 3 4 5
```

Notice that at step `i`, after swapping elements, we only need to apply **the exact same operation** to **the subvector** `v[1]` to `v[len-i]`.

We could do this by **recursion**.

Problem 2

The pseudo code for a recursive version of sorting is this:

Input: Vector `v` with length `len`.

Output: Vector `v` in ascending order.

Sort:

1. Find the index of the maximum element.
2. Swap the maximum with the `len`-th element (the last element in the **subvector**).
3. If `len` > 1
   - Sort rest of the vector by calling `sort(v[1:(len - 1)])`.

Create a new function called `sort_v2` implementing the pseudo code above, and test it.

Time complexity

```r
v <- c(5,3,2,1,2,4)
sort(v)  # length(v) = 6
```

```
[1] 1 2 2 3 4 5
```

after each swap, vector looks like

```
Iteration 0: 5 3 2 1 2 4
Iteration 1: 4 3 2 1 2 5
Iteration 2: 2 3 2 1 4 5
Iteration 3: 2 1 2 3 4 5
Iteration 4: 2 1 2 3 4 5
Iteration 5: 1 2 2 3 4 5
```

Sort 1.0 and Sort 2.0 are the **same algorithm** with different implementations, i.e., Loop vs. Recursion.

Clearly, as the length of the vector gets longer, our sorting algorithm is getting slower.

How much slower?

We need to call `find_max_idx` `len-1` times.

At iteration `i`, `find_max_idx` computes `len-i` comparisons.

- In total, we compute `len-1` + `len-2` ... + `1` comparisons.
- This can be written $\frac{\text{len}(\text{len}-1)}{2}$.
- Note that when `len` is large, the quadratic term (len^2) in $\frac{\text{len}(\text{len}-1)}{2}$ dominates.
- So we write $\frac{\text{len}(\text{len}-1)}{2} \propto \text{len}^2$, ignoring the other terms.
- So the cost grows quadratically with `len`!

That means that, if the sorting algorithm takes t seconds to run on a 1000 elements vector, it will take $4t$ seconds to run on an 2000 element vector.

We can say our sorting algorithm has a time complexity $O(\text{len}^2)$.

- Again, we only care about the dominating term as `len` increases.

Examples:

- $\text{len} + \sqrt{\text{len}} + 1$ is $O(\text{len})$.
- $\log \text{len} + \frac{\text{len}^3}{2}$ is $O(\text{len}^3)$.

Time complexity describes how computational time grows with the problem size.

Problem 3

Assume that each comparison performed by `find_max_idx` takes one second.

- **Question**: Suppose you are sorting a length 5 vector. How many seconds will it take to run the sorting algorithm?
- **Question**: Suppose you are sorting a length 10 vector. How many seconds will it take to run the sorting algorithm?

Problem 4 (optional, good for exam preparation)

The algorithm we introduced in the lecture is selection sort. A related algorithm is exchange sort. Can you implement exchange sort based on the following pseudo code?

Input: vector `v` and its length `len`

Output: The sorted vector `v`.

```
sort_exchange_recur(v)

    if len <= 1, return v.

    for i from 1 to len
        if v[i] > v[len]
            swap v[i] and v[len]

    v[1:(len - 1)] <- sort_exchange_recur(v[1:(len-
1)])
```

Problem 5 (optional, good for exam preparation)

Implement the non-recursive implementation of exchange sort.

Input: vector `v` and its length `len`

Output: The sorted vector `v`.

```
sort_exchange(v)

# Extra challenge: do not read the pseudo-code below!
for m from len to 2
    for i from 1 to m-1
        if v[i] > v[m],
            swap v[i] and v[m].
```